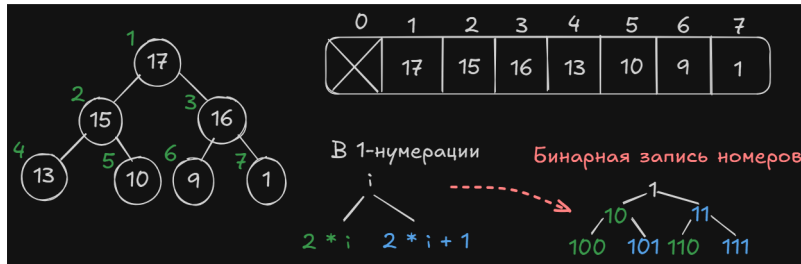


Куча (пирамида, heap)

Особенности

- Почти полное дерево, последний уровень заполняется слева направо подряд
- Значение вершины $\geq$ значений во всех потомках



Алгоритмы кучи

Просеивание вниз ($O(\log n)$)

- Если уменьшили элемент в куче: спускаемся в потомков (если они есть)
- Элемент $<$ потомка $\rightarrow$ swap (элемент, потомок)

```
void sift_down(int k) {
    int v = heap[k];
    while (2 * k < heap.size()) {
        if (2 * k + 1 < heap.size()) {
            if (heap[2 * k] <= heap[2 * k + 1] && heap[2 * k] <= v) {
                heap[k] = heap[2 * k];
                k = 2 * k;
            } else if (heap[2 * k + 1] <= v) {
                heap[k] = heap[2 * k + 1];
                k = 2 * k + 1;
            }
        } else if (heap[2 * k] <= v) {
            heap[k] = heap[2 * k];
            k = 2 * k;
        } else {
            break;
        }
    }
    heap[k] = v;
    return;
}
```

Просеивание вверх ($O(\log n)$)

- Если увеличили элемент в куче: поднимаемся вверх пока не дошли до корня или пока родитель меньше
- Элемент $>$ своего родителя $\rightarrow$ swap (элемент, родитель)

```
void sift_up(int k) {
    int v = heap[k];
    while (k > 1) {
        if (heap[k / 2] <= v) {
            break;
        }
        heap[k] = heap[k / 2];
        k = k / 2;
    }
    heap[k] = v;
    return;
}
```

Построение кучи ($O(n \log n)$)

- Добавляем новый элемент в конец, делаем sift_up

Построение кучи ($O(n - \log n)$)

- Начинаем с $n / 2$ и идём до 1, делая sift_down

Удаление элемента

1. Удаление из корня
 - Ставим значение последнего элемента в куче и делаем shift_down
2. Удаление не из корня
 - Ставим в вершину значение, большее значения в корне, shift_up . Свели к удалению из корня